

Anomaly Detection Results

Baseline Results

After running the notebook, copy the baseline results here:

Metric	Value
Total data points	1050
Anomalies injected	50
Anomalies detected by the model	53
True positives (correctly flagged)	40
False positives (normal data flagged)	13
False negatives (anomalies missed)	10
Precision	0.755
Recall	0.800
F1 Score	0.777

Scatter Plot Observations

The detected anomalies are mostly located outside the main cluster of normal sensor readings, particularly in regions with unusual pressure or flow values. The model also flags some points within the normal cluster, which explains the false positives, while some actual anomalies remain undetected.

Parameter Experiment

You changed `contamination` from `0.05` to `0.1`. Record the comparison below:

Metric	Default (0.05)	Changed (0.1)
Anomalies detected	53	105
True positives	40	40
False positives	13	65
Precision	0.755	0.381
Recall	0.800	0.800
F1 Score	0.777	0.516

Write one sentence explaining what changed when you increased contamination:

When contamination was increased from 0.05 to 0.1, the model flagged more points as anomalies, recall remained steady at 0.800 but also increasing false positives from 13 to 65 and reducing precision from 0.755 to 0.381.

Hyperparameter Experiments

Contamination Sweep

The contamination parameter controls what fraction of the data the model treats as anomalous.

Contamination	Anomalies Detected	Precision	Recall	F1 Score
0.01	11	1.000	0.222	0.361
0.03	32	1.000	0.640	0.780
0.05	53	0.736	0.780	0.757
0.1	105	0.381	0.800	0.516
0.2	210	0.190	0.800	0.308

Which contamination value gives the best F1 score?

Best contamination by F1 score: 0.03 (F1 = 0.780)

Tree Count Experiment

The `n_estimators` parameter controls how many trees the model builds. More trees means more "opinions" and more stable results, but slower training.

n_estimators	Precision	Recall	F1 Score	Training Time (sec)
50	0.698	0.740	0.718	0.114s
100	0.736	0.780	0.757	0.130s
200	0.717	0.760	0.738	0.349s
500	0.717	0.760	0.738	1.643s

At what point do more trees stop improving the results?

The results improve from 50 to 100 trees, but stop improving beyond 100 trees. Increasing the number to 200 or 500 trees does not improve the F1 score.

Does training time increase linearly with the number of trees?

No, training time does not increase linearly in these experiments. It increases from 0.130 seconds at 100 trees to 1.643 seconds at 500 trees, while the model's F1 score remains unchanged.

Sample Size Experiment

The `max_samples` parameter controls how much data each tree gets to see. Smaller samples make trees more diverse but potentially less accurate individually.

max_samples	Precision	Recall	F1 Score
auto	0.736	0.780	0.757
64	0.585	0.620	0.602
128	0.698	0.740	0.718
256	0.736	0.780	0.757

Which sample size gives the best F1 score?

Best max_samples by F1 score: auto (F1 = 0.757)

Best Configuration

Configuration	Precision	Recall	F1 Score
Default model	0.736	0.780	0.757
Your best model	1.000	0.640	0.780
Improvement	+0.264	-0.140	+0.023

Write 2-3 sentences analyzing the difference:

The best configuration, using a contamination value of 0.03, achieved an F1 score of 0.780, improving on the default model's 0.757. Precision increased from 0.736 to 1.000, meaning there were no false positives, while recall decreased from 0.780 to 0.640, meaning more actual anomalies were missed. Overall, the best configuration provides a better balance according to the F1 score, although it is less effective at catching every anomaly.

Model Comparison

LOF Default Results

Metric	LOF Default
Anomalies detected	53(40+13)
Precision	0.755
Recall	0.800
F1 Score	0.777

LOF Tuning (n_neighbors)

n_neighbors	Precision	Recall	F1 Score	Time(s)
10	0.755	0.800	0.777	0.007s
20	0.755	0.800	0.777	0.006s
50	0.755	0.800	0.777	0.012s

Which n_neighbors value gives the best F1 score?

Best n_neighbors by F1: 10 (F1 = 0.777)

Note:

All three values (10, 20, and 50) achieved the same F1 score of 0.777. Therefore, there is no single best value based on F1 score. However, `n\_neighbors=20` was the fastest in this experiment, with a training time of 0.006 seconds.

Head-to-Head Comparison

Metric	Isolation Forest (Best)	LOF (Best)	Winner
Precision	1.000	0.755	IF
Recall	0.640	0.800	LOF
F1 Score	0.780	0.777	IF
Training Time	0.141s	0.007s	LOF
Can score new data one at a time?	Yes	No	IF
Needs full dataset present?	No	Yes	IF

Your Recommendation

Which model should the Grand Marina deploy for real-time anomaly detection on their water system, and why? Write 3-5 sentences.

I recommend Isolation Forest for Grand Marina's real-time water system monitoring. Although LOF achieved slightly higher recall (0.800 compared with 0.640), Isolation Forest achieved a slightly higher F1 score (0.780 compared with 0.777) and perfect precision (1.000), meaning it produced no false positives in this experiment. This is important because false alarms can cause unnecessary responses from the night shift. Isolation Forest is also better suited to scoring new MQTT messages using an already-trained model, making it easier to integrate into the existing real-time dashboard. However, Grand Marina should consider whether missing some actual anomalies is acceptable, given the lower recall.

Overall winner by F1: Isolation Forest

Note: Even if LOF has slightly better F1, Isolation Forest is the better choice for real-time IoT because it can score new messages one at a time as they arrive from the MQTT broker.

Optional: One-Class SVM Results

Metric	One-Class SVM
Anomalies detected	54(33+21)
Precision	0.611
Recall	0.660
F1 Score	0.635
Training Time	0.005s